



Part of the AgentCon World Tour by the Global AI Community

AI Agents Developer Conference

agentcon.city/rome



Next-Gen AI Assistants: Open, Interoperable, Extensible with WayFlow

Corrado De Bari



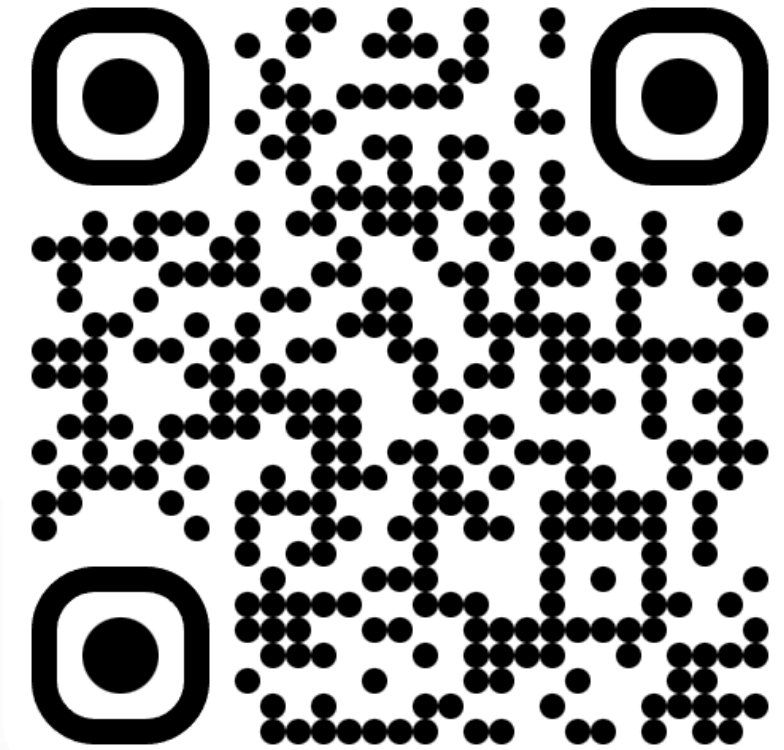
Corrado De Bari

agentcon.city/rome

Made Possible by



Join the AI Conversation
on Discord



gaic.io/discord

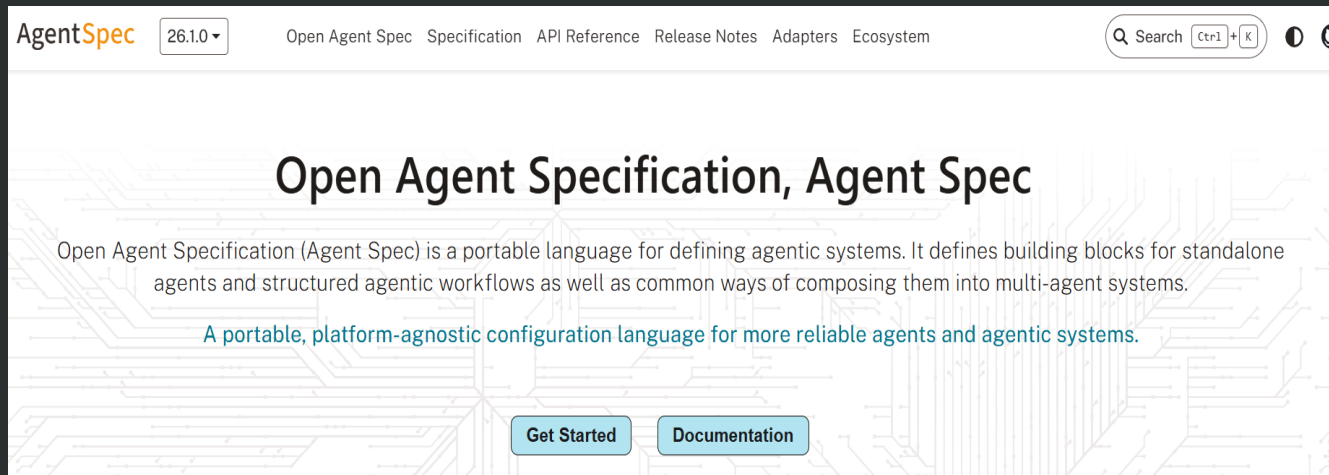
Safe harbor statement

The following is intended to outline our general product direction. It is intended for information purposes only, and may not be incorporated into any contract. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described for Oracle's products may change and remains at the sole discretion of Oracle Corporation.

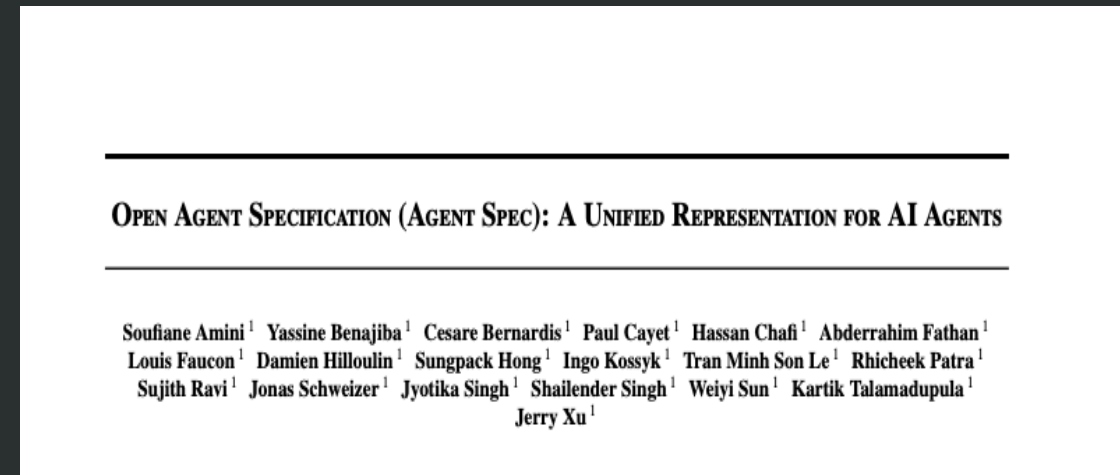
Open Agent Specification & Wayflow

Open Agent Specification

Maybe we need a way to share how we represent our agent ?
(It is not that much code, anyway)



oracle.github.io/agent-spec/



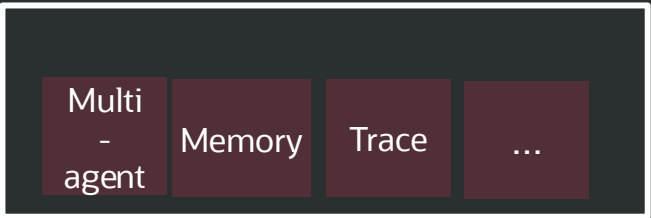
arxiv.org/pdf/2510.04173

Customer Problems & Business impact

Build, Scale, and Maintain AI Agents

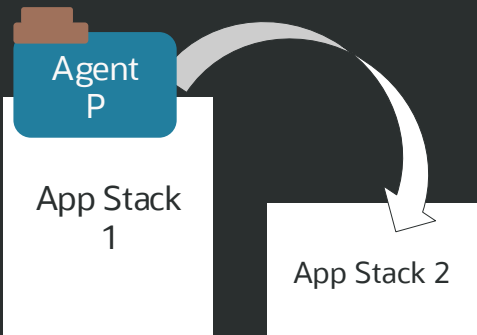
Common Abstraction

Flow, Multi-Agent, Chains, Orchestration ...
Separate substrate (python) code from actual agent logic



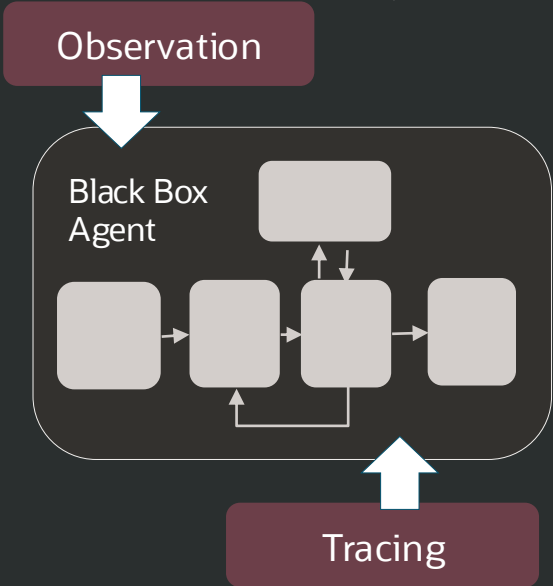
Reusability

Different teams and organizations use different frameworks
When migrating to new software stack



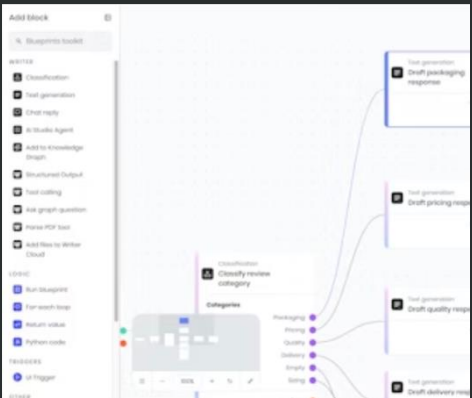
Framework-Agnostic Tooling

Third-party toolsets need to be frame-agnostic.
Tools for injecting observability, UI/UX event response, traceability



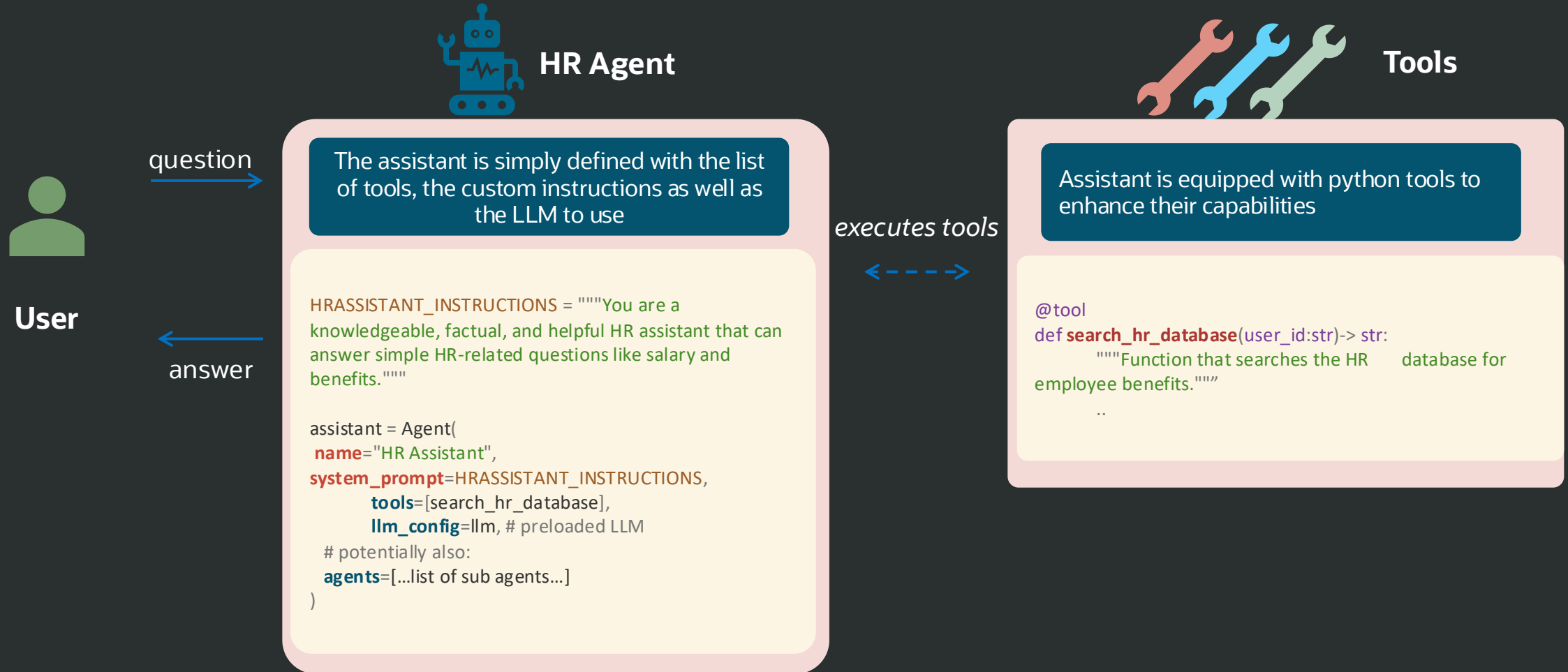
Authoring and Generation

Authoring tools (Low-code UI) also value frame-agnostic nature of Open Agent Spec
Generations tools can get benefits from Open Agent Spec



Agent Spec - Examples

Tool Calling Agent – code example



Agent Spec - Examples

Tool Calling Agent – JSON representation

Basic agent requires:

- LLM configuration
- System prompt
- Tool configurations

Advanced:

- Calling fixed flows as tools
- Planner component
- etc

Basic agent
configuration

LLM specification

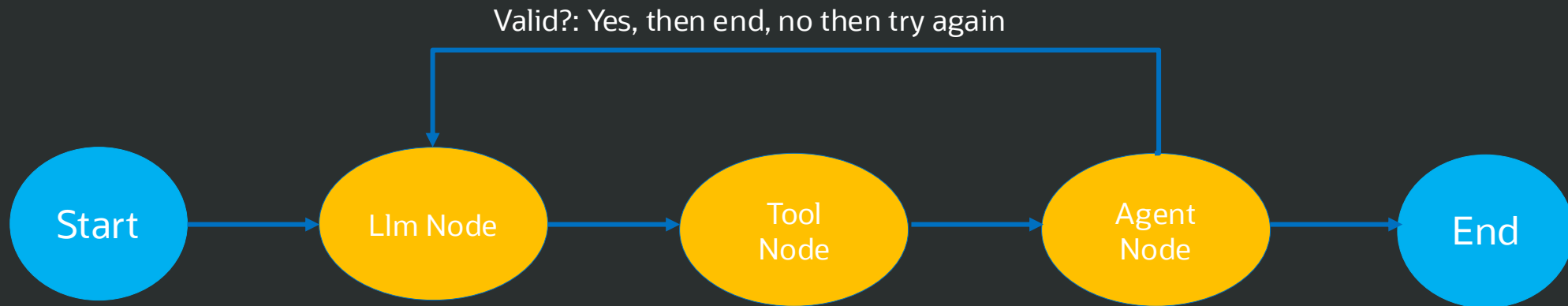
Tool specification

```
{
  "component_type": "Agent",
  "id": "67287cab-8068-4a3a-9a14-4783adacc2b5",
  "name": "HR Agent",
  "description": "HR Agent",
  "inputs": [],
  "outputs": [],
  "llm_config": {"$component_ref": "vllm"},
  "system_prompt": "You are a knowledgeable, helpful HR assistant that can answer simple HR-related questions like salary and benefits.",
  "tools": [{" $component_ref": "search_hr_database"}],
  "metadata": {},
  "$referenced_components": {
    "vllm": {
      "component_type": "OCI GenAI LLM Config",
      "id": "llamallm",
      "name": "llamallm",
      "model_id": "oci...123"
    },
    ...
  },
  "search_hr_database": {
    "component_type": "ServerTool",
    "id": "search_hr_database",
    "name": "search_hr_database",
    "description": "Gets information from the HR database",
    "inputs": [{"title": "query", "type": "string"}],
    "outputs": [{"title": "results", "type": "array", "items": {"type": "string"}}]
  }
}
```

Agent Spec - Examples

Flows

A fixed flow specifies the different steps (nodes) that can be run and transitions between them.
Control and data flow is constrained.
Some nodes might use LLMs, some nodes not (e.g. do REST API call).
(Loops are possible)

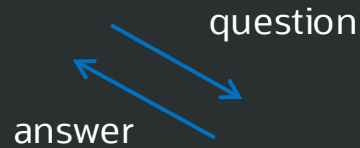


Agent Spec - Examples

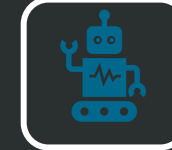
Flow – code example



User



Tool Node



LLM Node

The Flow is easily defined with nodes and transitions between the nodes

```
start_node = StartNode(name="start", inputs=[query_property])
tool_node = ToolNode(
    name="hr_lookup_node",
    tool=search_hr_database,
    outputs=[results_property]
)
llm_node = LlmNode(
    name="llm_answer_step",
    llm_config=llm_config,
    prompt_template="Answer the user given the question {{ query }} with the given context:
    {{ hr_context }}",
    inputs=[query_property, hr_context_property],
    outputs=[answer_property],
)
end_node = EndNode(name="end", outputs=[answer_property])
```

```
control_flow_edges = [
    ControlFlowEdge(name="tool_to_llm", from_node=tool_node, to_node=llm_node),
    ...
]

data_edges = [
    DataFlowEdge(name="query_data_edge", source_node=start_node,
    source_output="query", destination_node=tool_node, destination_input="query"), ...
]

flow = Flow(
    name="HR Flow",
    start_node=start_node,
    nodes=[start_node, tool_node, llm_node, end_node],
    control_flow_connections=control_flow_edges,
    data_flow_connections=data_flow_edges,
)
```

Agent Spec - Examples

Flow – JSON representation

Fixed flows require:

- List of nodes
- Control/data flow definition

Steps require:

- Inputs/outputs definition
- Per node configuration

Flow
information

Nodes in
flow

Control flow
definition

Data flow
definition

```
{
  "component_type": "Flow",
  "id": "1a144254-6296-47d9-bbf5-41d4128d8383",
  "name": "HR Flow",
  "inputs": [{"title": "query", "type": "string"}],
  "outputs": [{"title": "answer", "type": "string"}],
  "start_node": {"$component_ref": "start_node"},
  "nodes": [
    {
      "component_type": "StartNode"
    },
    {
      "component_type": "LlmNode",
      "prompt_template": "Answer the user ...",
      "llm_config": {"$component_ref": "vllm"},
      "name": "llm_answer_step",
      "outputs": [{"title": "answer", "type": "string"}]
    }
  ],
  "control_flow_connections": [{
    "component_type": "ControlFlowEdge",
    "from_node": {"$component_ref": "start_node"},
    "name": "tool_to_llm",
    "to_node": {"$component_ref": "tool_node"}
  }],
  "data_flow_connections": [{
    "component_type": "DataFlowEdge",
    "destination_input": "query",
    "destination_node": {"$component_ref": "start_node"},
    "name": "query2_data_edge",
    "source_node": {"$component_ref": "tool_node"},
    "source_output": "query"
  }
  ]
}
```

WayFlow: From Agent Spec definition to execution

Reference Implementation

Export to Agent Spec JSON/YAML is two lines of code

The assistant is simply defined with the list of tools, the custom instructions as well as the LLM to use

```
MTASSISTANT_INSTRUCTIONS = """You are an assistant for helping with  
math homework.  
"""
```

```
multiplication_tool = ServerTool(  
    name="multiplication_tool",  
    description="Tool that allows to compute  
multiplications",  
    inputs=[IntegerProperty(title="a"),  
            IntegerProperty(title="b")],  
    outputs=[IntegerProperty(title="product")],  
)  
...
```

```
assistant = Agent(  
    name="Math homework assistant",  
    system_prompt=MTASSISTANT_INSTRUCTIONS,  
    tools=[multiplication_tool],  
    llm_config=llm, # preloaded LLM  
)
```

```
from pyagentspec.serialization import  
    AgentSpecSerializer
```

```
EXPORTED_CONFIG =  
    AgentSpecSerializer().to_yaml(assistant)
```

WayFlow: From Agent Spec definition to execution

Reference Implementation

- Install the Agent Spec adapter for WayFlow.
- Define the execution of tools in a tool registry.
- Load an Agent Spec configuration using the WayFlow adapter.
- Run the assistant and interact with it.

Import as Wayflow agent and run

```
..
from wayflowcore.tools import ServerTool

multiplication_tool = ServerTool(
    name="multiplication_tool",
    description="Tool that allows to compute
                multiplications",
    ..
    func=lambda a, b: a*b )

tool_registry = {
    "multiplication_tool": multiplication_tool
}

from wayflowcore.agentspec import AgentSpecLoader

loader = AgentSpecLoader(tool_registry=tool_registry)
assistant = loader.load_yaml(EXPORTED_CONFIG)

...
conversation = assistant.start_conversation()
while True:
    ...
    conversation.execute()
    messages = conversation.get_messages()
    ...
```

WayFlow- Serve

Deploying Easily Agents from Agent Spec configs or code

- WayFlow Serve is usable as CLI or as Python package; ideal for cloud deployments
- Can serve Agents with an OpenAI Responses or A2A API
 - Enables interesting compositions: use your Agent just like a LLM in other Agents!

The Agent can be deployed leveraging the WayFlow server CLI command by providing an Agent Spec config

```
$ wayflow serve \  
--api openai-responses \  
--agent-config hr_agent.json \  
--agent-id hr-assistant \  
--server-storage oracle-db \  
--datastore-connection-config oracle_conn.yaml \  
--setup-datastore yes
```

OR

Through code (useful if needs Python tools)

```
HRASSISTANT_INSTRUCTIONS = """You are a knowledgeable,  
factual, and helpful HR assistant that can answer simple HR-  
related questions like salary and benefits."""
```

```
assistant = Agent(  
    name="HR Assistant",  
    system_prompt=HRASSISTANT_INSTRUCTIONS,  
    tools=[search_hr_database],  
    llm_config=llm, # preloaded LLM  
)
```

```
from wayflowcore.agentserver.server import  
OpenAIResponsesServer
```

```
server = OpenAIResponsesServer()  
server.serve_agent("HR Assistant", assistant)  
app = server.get_app()  
if __name__ == "__main__":  
    server.run(host="127.0.0.1", port=3000)
```

The Agent is up & running!

```
$ curl  
http://127.0.0.1:3000/v1/models  
{"object": "list", "data": [{"id": "hr-  
assistant" ...}]}
```


Flow Demos

Open Agent Specs – Representation and Runtime adapters

Enabling deployment to the right platform

Agent Spec is purely a representation of an agent or agentic workflow

A runtime adapter makes Agent Spec components executable by implementing the behavioral logic defined in the representation

Developers can prototype and develop on one platform (e.g. LangGraph) and deploy on another

Agent Spec Representation (e.g. RAG Agent)

```
{
  "component_type": "Agent",
  "name": "adaptive_expert_agent",
  "inputs": [{"title": "domain_of_expertise"
...}],
  "outputs": [],
  "llm_config": {
    "component_type": "VllmConfig",
    "name": "Llama 3.1 8B instruct",
    "url": "http://llama.deployment.url",
    "model_id": "Meta-Llama-3.1-8B-Instruct"
  },
  "system_prompt": "You are an expert in {{domain_of_expertise}}. Please help the users with their requests.",
  "tools": [{
    "component_type": "ServerTool",
    "name": "rag_tool",
    "description": "Tool that performs RAG",
    "inputs": [{"title": "query", ...}],
    "outputs": [{"title": "results", ...}]
  }],
  "agentspec_version": "25.4.1"
}
```

LangGraph Adapter

WayFlow Adapter

CrewAI*

Microsoft Agent
Framework Adapter*

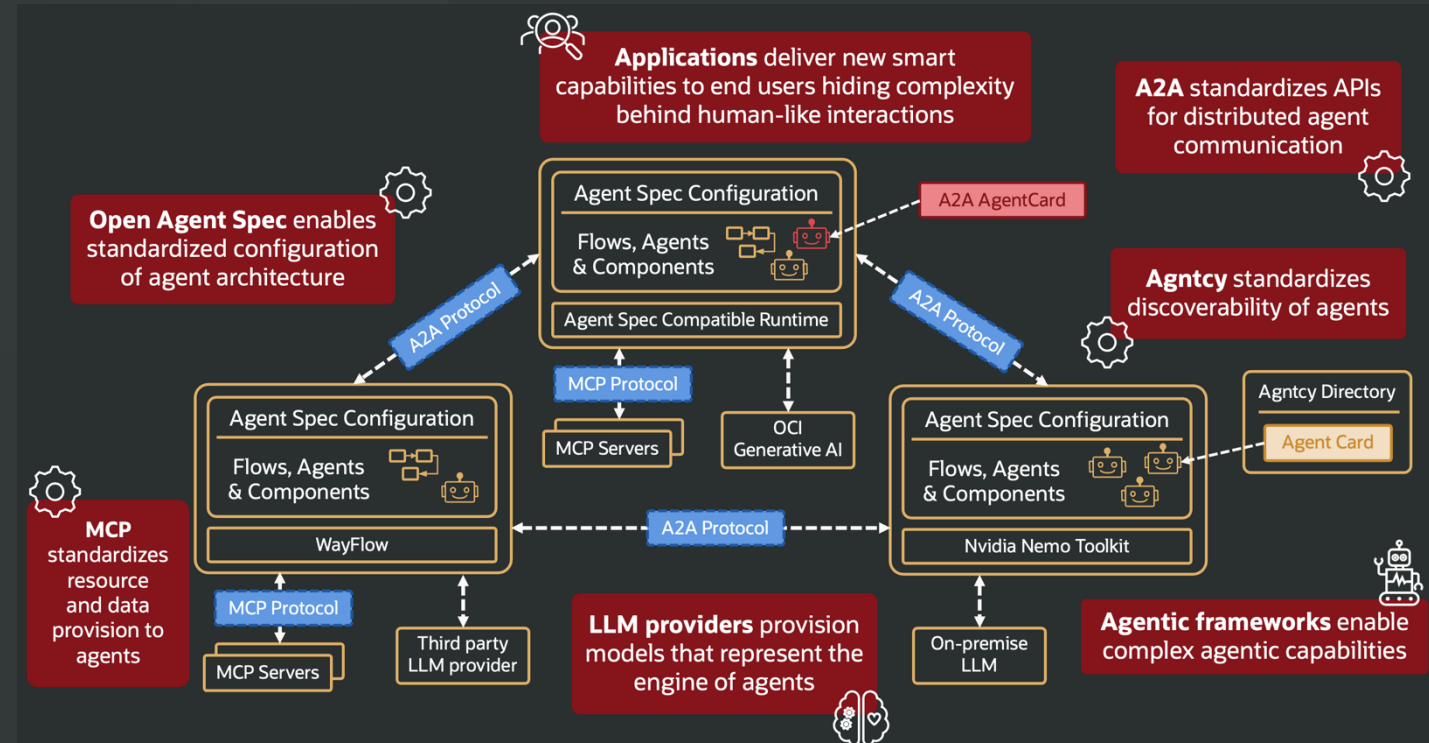
OpenAI Agents SDK*

Agent Spec: cross-framework Portability DEMO

LangGraph → Agent Spec
Agent Spec → LangGraph

Relationship to other protocols (MCP, A2A)

- **Agent Spec** defines each Agent instance **behaviorally**.
- **MCP** defines Agent **interacting** with tools
- **A2A** defines Agent **communicating** with another agent
- Agent Spec enables integration of MCP tools and A2A Agents



Framework Agnostic UI Observability Injection

Components for Agent Traces

Open Agent Specification Tracing (Agent Spec Tracing)

Overview

Open Agent Specification Tracing (short: Agent Spec Tracing) is an extension of Agent Spec that standardizes how agent and flow executions emit traces. It defines a unified, implementation-agnostic semantic for:

- Events: structured, point-in-time facts.
- Spans: time-bounded execution contexts that group events.
- Traces: trees of spans that represent an end-to-end execution.
- SpanProcessors: pluggable hooks to consume spans and events (e.g., export to UIs, tracing backends, or logs).

Example

```
from pyagentspec.tracing.trace import Trace
from wayflowcore.agentspec import AgentSpecLoader
from wayflowcore.agentspec.tracing import AgentSpecTracingEventListener
from wayflowcore.events.eventlistener import register_event_listeners
from openinference_spanprocessor import ArizePhoenixSpanProcessor

agent_json = read_json_file("my/agentspec/agent.json")
processor = ArizePhoenixSpanProcessor(mask_sensitive_information=False,
                                     project_name="agentspec-tracing-test")

with register_event_listeners([AgentSpecTracingEventListener()]):
    with Trace(name="agentspec_wayflow_demo", span_processors=[processor])
    as trace:
        agent = AgentSpecLoader().load_json(agent_json)
        conversation = agent.start_conversation()
        status = conversation.execute()
```

Source: <https://oracle.github.io/agent-spec/26.1.0/agentspec/tracing.html>

Agent Spec Collaborations

AI agents easier to discover, share, and run across frameworks



AGNTCY

AGNTCY supports Agent Spec metadata in OASF Agent records to enable developers to specify how to consume a deployed agent or instantiate new agents with minimal effort.

agent discovery

agent deployment

Source:

<https://agntcy.org/supporters>

<https://outshift.cisco.com/blog/ai-ml/open-agent-specification-support-and-deployment-data>

Framework Agnostic UI tool Integration



AG-UI Protocol

AG-UI

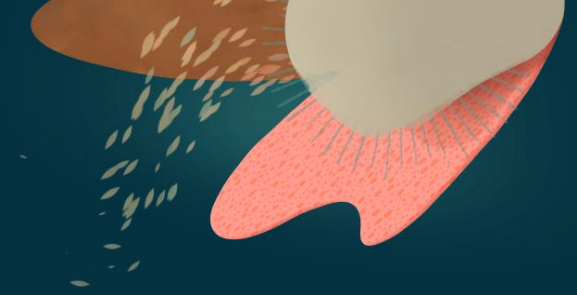
The integration of AG-UI with Open Agent Spec allows developers to easily connect any AG-UI-compatible frontend with Agent Spec, enabling users to load any Agent Spec config file, choosing a supported runtime to execute it.

agentic frontends

Source:

<https://www.copilotkit.ai/blog/oracle-adopts-ag-ui-protocol-for-agent-spec>

Introducing the Private Agent Factory



AI Database > Technologies >

Oracle AI Database Private Agent Factory

Oracle AI Database Private Agent Factory is a no-code platform to build custom AI agents and workflow automations that connect safely to your enterprise data and tools, then deploy in minutes with enterprise-grade security, performance and governance. Build and deploy enterprise ready AI agents and workflow automations using Knowledge based AI agents, Data Analysis Agent, AI workflow automation, Security and governance, No-code Agent Builder and Models and interoperability.

Download: <https://www.oracle.com/database/technologies/private-agent-factory-downloads.html>

Agent Builder

ORACLE[®] AI Database Private Agent Factory

STUDIO

Components

My workflows / AI Agent

Save

New Flow

Playground

Publish

SQL query

Execute SQL Query (only SELECT statements are allowed for security reasons).

Select database

Applied AI Datasets

Include columns

Query

select * from SALES order by TIME limit 10;

JSON

Message

LLM

Send a text message to an LLM model. To configure a new one, go to LLM management and add a new LLM to use.

Select LLM to use

Prompt

Receiving input

Temperature

0.01

Message

Email output

Send an email notification.

Recipient email(s)

New Trend Detected

Email subject

Workflow Response

Email body

Status

PRE-BUILT AGENTS

UTILITIES

SETTINGS

React Flow

Copyright © 2025 Oracle and/or its affiliates. All rights reserved.

About Oracle

Contact Us

Terms Of Use

Your Privacy Rights



Import Agent Spec as a Custom flows

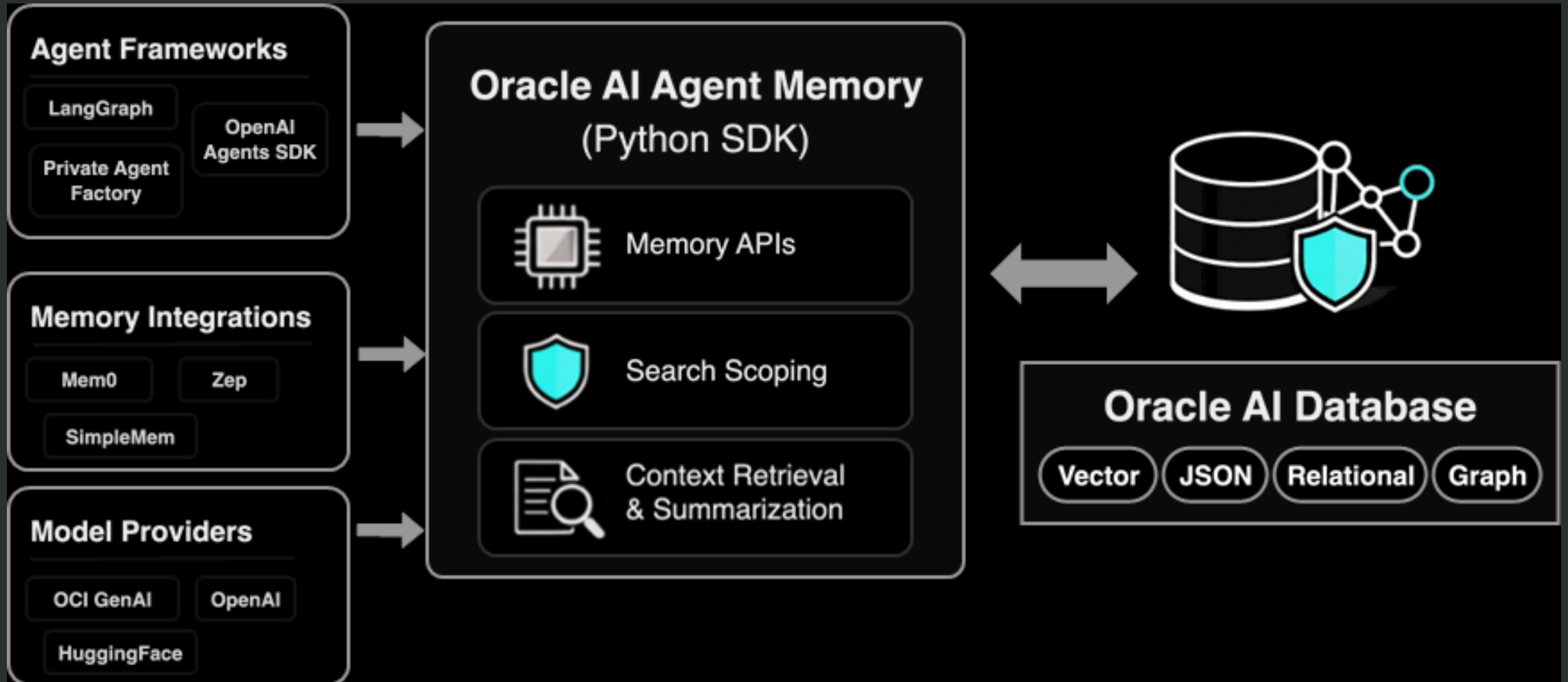
The screenshot displays the Oracle Private AI Agent Studio interface. On the left, a sidebar menu lists various options: STUDIO (Getting started, Template gallery, My custom flows), PRE-BUILT AGENTS (Knowledge agent, Data analysis agent), UTILITIES (Agent builder, Prompt lab, Datasets), and SETTINGS (Data sources, SSO settings, User management). The 'My custom flows' option is highlighted. The main area shows a chat interface for the 'Bug Agent', which is described as 'Agent to help with bug diagnostics'. The chat history shows a message from 'echelon_admin@oracle.com (You)' asking 'Who is assigned to bug 38182376?'. The agent's response is 'Processing..'. At the bottom, a notification states 'New chat created successfully.'.



Oracle AI Agent Memory

Collaterals for agent development

Oracle AI Agent Memory



Use Agent Memory with WayFlow example

```
from oracleagentmemory.core.oracleagentmemory
    import OracleAgentMemory

memory = OracleAgentMemory(
    connection=db_pool,
    embedder=embedder,
    llm=llm,
    schema_policy= SchemaPolicy.CREATE_IF_NECESSARY
)

memory_thread = memory.create_thread(
    thread_id="wayflow_memory_demo",
    user_id=user_id,
    agent_id=agent_id,
)
```

```
@tool
async def search_memory(
    query: Annotated[str,
        "Question to search in Oracle Agent
        Memory"],)
    -> Annotated[str, "Top matching memory content"]:
    """Search Oracle Agent Memory for durable user
    facts relevant to the current request."""

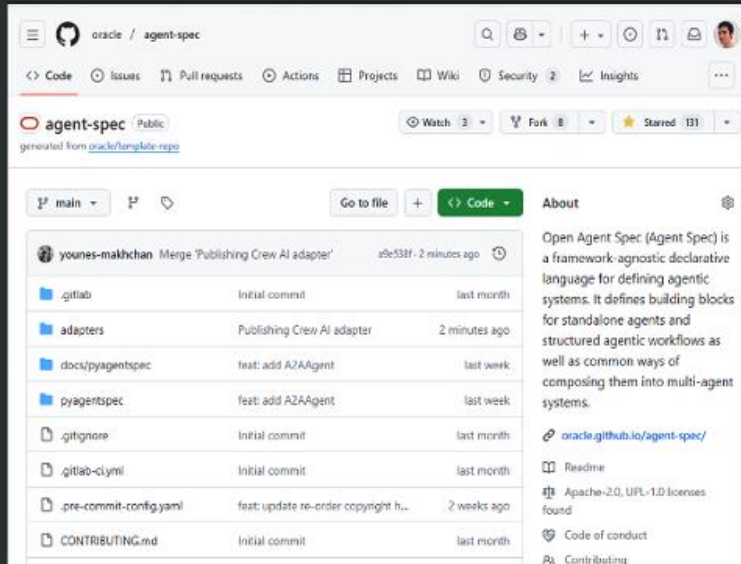
    results = await memory.search_async(
        query=query,
        user_id=user_id,
        agent_id=agent_id,
        max_results=1,
        record_types=["memory", "messages"])
    if not results:
        return "No relevant memory found."

    return results[0].content
```

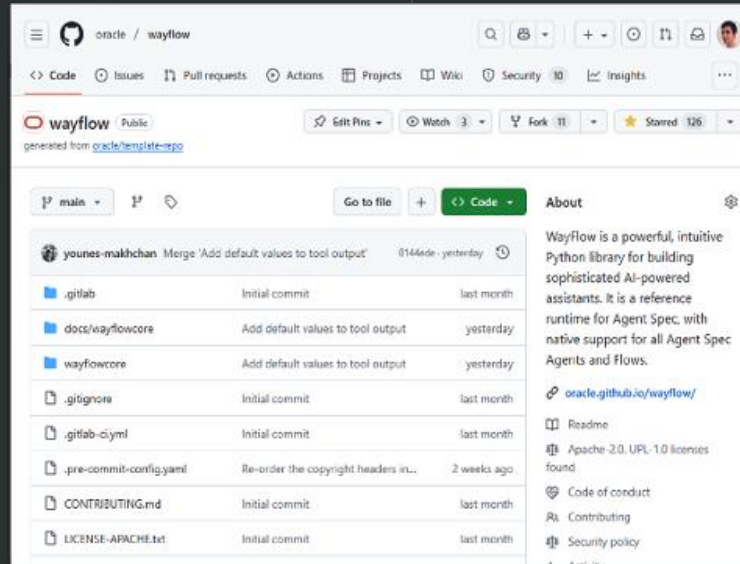
Source: <https://docs.oracle.com/en/database/oracle/agent-memory/26.4/agmea/int-wayflow.html>

Takeaways

```
$ pip install pyagentspec
```



```
$ pip install wayflowcore
```



```
$ pip install oracleagentmemory
```

Oracle AI Database > Unified Agent Memory Core >

Oracle AI Agent Memory

Oracle AI Agent Memory is a Python library that helps developers build agents that can remember, reason, and act with enterprise context. Part of Oracle's Unified Agent Memory Core, it gives agents a persistent memory layer for conversations, facts, preferences, relationships, and business context. Agent memory empowers every interaction to become part of a more useful, connected intelligence.

Explore Unified Agent Memory Core

Python Library

Open Agent Specification
github.com/oracle/agent-spec

WayFlow (runtime for AgentSpec)
github.com/oracle/agent-spec

Oracle AI Agent Memory
www.oracle.com/database/ai-agent-memory

Oracle AI Database 26ai **Free** Container Image : <https://container-registry.oracle.com/>



Thank you, and join me on LinkedIn!

GitHub: <https://github.com/corradodebari>

ORACLE